TCC Writing Plan — Adaptive Main-Thread vs Worker Scheduling

2026-11-02

The thesis in one paragraph

Web apps that process tabular data (sorting, filtering, grouping in TanStack Table) block the main thread and cause jank. Moving that work to a Web Worker has a fixed cost: message cloning, scheduling, and transfer back. The best choice depends on data size and on the device's current load.

This work proposes a drop-in bundle that does three things:

- Collects runtime signals: Long Animation Frames, Compute Pressure, memory, and its own timings.
- Learns a per-device cost model.
- Decides per operation whether to run on the main thread or in a worker.

The expected result is that the main-thread/worker crossover point moves with device load. The adaptive scheduler should track it better than always-main, always-worker or a fixed threshold.

Research question: can a runtime, telemetry-driven scheduler reduce main-thread blocking (TBT, INP) for client-side table computation without increasing time-to-result, compared with static strategies?

Chapters and page budget

The body has seven chapters totaling about 28 pages. Front matter and references are not counted. Results and implementation get the most room, since that is where the contribution shows.

#	Chapter	Pages	What goes in it
1	Introduction	2–3	The jank problem in data-heavy UIs; motivation; research question; general and specific objectives; contributions; document structure
2	Theoretical background	5–6	Browser event loop and rendering pipeline (16 ms frame, 50 ms long task); Web Workers; postMessage and structured clone, transferables, SharedArrayBuffer + Atomics and cross-origin isolation; performance APIs (Long Tasks, LoAF, Event Timing/INP, Compute Pressure, memory, Resource Timing); TanStack Table as a headless library and its row models
3	Related work	2–3	Worker offloading libraries (Comlink, Partytown, workerize); adaptive/offloading schedulers in mobile and edge computing; Web performance studies on main-thread blocking. End with a gap table showing what none of them do: a runtime, device-aware decision for UI computation
4	Proposed approach	4–5	Architecture (bundle, telemetry collector, cost model, scheduler, worker, table adapter); columnar data layout; the shared pure compute function; cost model equations (k·n·log n, fixed worker overhead, EWMA); decision rule with blocking budget and exploration rate; bundle vs. extension trade-offs
5	Implementation	4–5	Stack (Vite, React, TanStack Table); message protocol with reqId and stale-response handling; transfer of Int32Array indices; manualSorting/manualFiltering integration; CSP and same-origin limits for workers; short code excerpts only
6	Experiments and results	6–7	Setup (devices, browsers, throttling 1x/4x/6x, background load); data sizes 100 to 500k rows; comparator types; the four strategies; metrics (TBT, INP, time-to-result, decision accuracy); results charts; crossover analysis; threats to validity
7	Conclusion	1–2	Answer to the research question with numbers; limitations (Chromium-only APIs, no direct CPU counters); future work (extension telemetry, SharedArrayBuffer path, other libraries, WebGPU)

Front matter (cover, abstract in Portuguese and English, lists) and references follow your university's ABNT template. They sit outside the 25–30 pages unless your course counts them.

Figures and tables to produce

Around 10 exhibits carry the argument, and each one should earn its page space.

Exhibit	Chapter	Shows
Event loop and frame timeline	2	Where a long task blocks input and rendering
Main thread ↔ worker message flow	2	Clone vs. transfer vs. shared memory
Comparison of communication mechanisms	2	Cost, zero-copy, requirements (table)
Related work gap table	3	Which features each prior work covers
System architecture diagram	4	Bundle components and data flow
Decision flowchart	4	How the scheduler picks main vs. worker
Time vs. row count, main and worker, per load level	6	The crossover point and how it moves (the key chart)
TBT and INP per strategy	6	Adaptive vs. static strategies
Decision accuracy per load level	6	How often the scheduler chose right
Overhead breakdown (clone, compute, transfer back)	6	Where worker time goes

Writing order

Write in the order the work produces material, not in chapter order. The introduction and abstract come last, once you know the result.

1. **Theoretical background (ch. 2)** while you build. Reading the specs is research you need anyway.
2. **Proposed approach (ch. 4)** as soon as the architecture stabilizes. Draw the architecture diagram early.
3. **Implementation (ch. 5)** alongside the code. Note decisions and problems (CSP, stale responses) as they happen.
4. **Experiments (ch. 6).** Write the setup section before running benchmarks, so the protocol is fixed, then add results.
5. **Related work (ch. 3).** This is easier once you know exactly what your contribution is.
6. **Conclusion, introduction, abstract** last.

Practical tips:

- Save every benchmark run as CSV with device, browser, throttling level and commit hash, so the charts can be regenerated from these.
- Run each configuration several times and report median and spread, not a single run.
- Keep code in the text to short excerpts, and put the repository link in an appendix.
- Confirm with your advisor whether the 25–30 pages include front matter, and which ABNT template your course uses.